

Reviewer Pack — SOS Refutation Degree Bounded by Clause Density for Random 3...

Entry 52c162cccd9b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 01:51:22 UTC

SOS Refutation Degree Bounded by Clause Density for Random 3-SAT

Entry ID: 52c162cccd9b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 01:51:22 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): 3-SAT Instances

Statement:

For a random 3-SAT instance with n variables and m clauses, the minimal SOS degree required to refute it is $\Theta(\sqrt{m})$.

Rationale (proposer's reasoning):

The SOS hierarchy's refutation degree captures the algebraic complexity of constraint systems. By linking it to clause density, we expose geometric structure in SAT solving that could yield new lower bounds for refutation algorithms.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2f2ae0ebd1b7e228

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            A[j] = [A[j][k] - factor * A[i][k] for k in range(n)]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i + 1, n))) / A[i][i]
    return x

def poly_to_matrix(poly, variables):
    n = len(variables)
    m = len(poly)
    A = [[0] * n for _ in range(m)]
    b = [0] * m
    for i, p in enumerate(poly):
        terms = p.split('*')
        for term in terms:
            if '!' not in term:
                continue
            var = term[1:-2]
            A[i][variables.index(var)] += 1
        b[i] = -1
    return A, b

def sos_degree(poly, variables):
    n = len(variables)
    m = len(poly)
    A, b = poly_to_matrix(poly, variables)
    try:
        x = gaussian_elimination(A, b)
        return max(abs(x_i) for x_i in x)
    except Exception as e:
        return float('inf')

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

n = random.randint(5, 40)
m = 3 * n
variables = [f'x{i}' for i in range(n)]
clauses = ['!(x{0}*x{1}*{2})'.format(*random.sample(variables, 3)) for _ in range(m)]
poly = [c.replace('!', '') for c in clauses]
degree = sos_degree(poly, variables)
expected_degree = math.isclose(degree, math.sqrt(m), rel_tol=1e-2)
return {
    "metric_name": "SOS Degree",
    "metric_value": degree,
    "instances_tested": 1,
    "conjecture_holds": expected_degree,
    "counterexample": "" if expected_degree else f"Expected degree {math.sqrt(m)}, got {degree}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 1 for i in range(5, 30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}}}")
        results.append(result)

    mean_degree = sum(r['metric_value'] for r in results) / len(results)
    std_dev = math.sqrt(sum((r['metric_value'] - mean_degree)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_degree} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_degree} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\", first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

569495584, got inf"}
TRIAL: {"seed": 593, "metric_name": "SOS Degree", "metric_value": inf, "instances_tested": 1, "conjecture_holds": false}
TRIAL: {"seed": 631, "metric_name": "SOS Degree", "metric_value": inf, "instances_tested": 1, "conjecture_holds": false}
TRIAL: {"seed": 677, "metric_name": "SOS Degree", "metric_value": inf, "instances_tested": 1, "conjecture_holds": false}
TRIAL: {"seed": 727, "metric_name": "SOS Degree", "metric_value": inf, "instances_tested": 1, "conjecture_holds": false}
TRIAL: {"seed": 773, "metric_name": "SOS Degree", "metric_value": inf, "instances_tested": 1, "conjecture_holds": false}
TRIAL: {"seed": 821, "metric_name": "SOS Degree", "metric_value": inf, "instances_tested": 1, "conjecture_holds": false}
TRIAL: {"seed": 877, "metric_name": "SOS Degree", "metric_value": inf, "instances_tested": 1, "conjecture_holds": false}
TRIAL: {"seed": 929, "metric_name": "SOS Degree", "metric_value": inf, "instances_tested": 1, "conjecture_holds": false}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0e7c05f5.py", line 93, in <module>
    first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0e7c05f5.py", line 93, in <genexpr>
    first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

parse_fail | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	68726
2	preregistration	ollama_remote	qwen3:8b	0	0	24240
3	novelty	ollama_remote	qwen3:8b	0	0	22872
4	novelty	ollama_remote	qwen3:8b	0	0	16352
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16327
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10716
7	judge	ollama_remote	qwen3:8b	0	0	23764

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 182996 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/52c162cccd9b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/52c162cccd9b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/52c162cccd9b.tar.gz` (if generated)